

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ
ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ им. В. Г. ШУХОВА»
(БГТУ им. В.Г. Шухова)**

Кафедра программного обеспечения вычислительной техники и автоматизированных
систем

Лабораторная работа №15

по дисциплине: Основы программирования

тема: **«Создание библиотеки для работы с многомерными массивами»**

Выполнил: студент группы ПВ-221
Дорошенко Владимир Юрьевич

Проверили:
ст. преп. Притчин Иван Сергеевич
асс. Черников Сергей Викторович

Белгород 2023 г.

Цель работы: закрепление навыков создания библиотек, структур;
получение
навыков работы с многомерными массивами.

Содержание отчета:

- 1) Тема лабораторной работы.
- 2) Цель лабораторной работы.

3) Решения задач:

- Текст задания.
- Исходный код (в том числе и тестов).

Требования:

Выполните автоматизированное тестирование разработанной библиотеки

Функции:

2. В библиотеке `matrix` реализуйте функции для размещения в динамической памяти матриц:

```
#include <stdio.h>
#include <stdlib.h>
#include <assert.h>
#include "matrix.h"
#include <stdbool.h>
#include <string.h>

//2

Matrix getMatrix(int nRows, int nCols) {
    int **values = (int **) malloc(sizeof(int *) * nRows);
    for (int i = 0; i < nRows; i++) {
        values[i] = (int *) malloc(sizeof(int) * nCols);
    }

    return (Matrix) {values, nRows, nCols};
}

Matrix *getArrayOfMatrices(int numMatrix, int nRows, int nCols) {
    Matrix *m = (Matrix *) malloc(sizeof(Matrix) * numMatrix);

    for (int i = 0; i < numMatrix; i++) {
        m[i] = getMatrix(nRows, nCols);
    }

    return m;
}

void freeMatrix(Matrix m) {
    for (int i = 0; i < m.nRows; i++) {
        free(m.values[i]);
    }

    free(m.values);
}

void freeMemMatrices(Matrix *m, int numMatrix) {
    for (int i = 0; i < numMatrix; i++) {
        freeMatrix(m[i]);
    }

    return free(m);
}
```

3. В библиотеке matrix реализуйте функции для ввода и вывода матриц:

```
//3
void inputMatrix(Matrix m) {
    for (int i = 0; i < m.nRows; i++) {
        for (int j = 0; j < m.nCols; j++) {
            scanf("%d", &m.values[i][j]);
        }
    }
}

void outputMatrix(Matrix m) {
    for (int i = 0; i < m.nRows; i++) {
        for (int j = 0; j < m.nCols; j++) {
            printf("%d ", m.values[i][j]);
        }

        printf("\n");
    }
}

void inputMatrices(Matrix *m, int n) {
    for (int i = 0; i < n; i++) {
        inputMatrix(m[i]);
    }
}

void outputMatrices(Matrix *m, int n) {
    for (int i = 0; i < n; i++) {
        outputMatrix(m[i]);
        printf("\n");
    }
}
```

4. В библиотеке matrix реализуйте функции для обмена строк и столбцов:

```
//4
void swapWithPointer(int **a, int **b) {
    int *t = *a;
    *a = *b;
    *b = t;
}

void swapRows(Matrix m, int i1, int i2) {
    assert(i1 >= 0 && i1 < m.nRows);
    assert(i2 >= 0 && i2 < m.nRows);
    swapWithPointer(&m.values[i1], &m.values[i2]);
}

void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}

void swapColumns(Matrix m, int j1, int j2) {
    for (int i = 0; i < m.nRows; i++) {
        swap(&m.values[i][j1], &m.values[i][j2]);
    }
}
```

5. В библиотеке matrix реализуйте функции для упорядочивания строк и столбцов:

```
//5
int getSum(int *a, int n) {
    int sum = 0;
    for (int i = 0; i < n; i++) {
        sum += a[i];
    }

    return sum;
}

int getMin(int *m, int n) {
    int min = m[0];

    for (int i = 0; i < n; i++) {
        if (min > m[i]) {
            min = m[i];
        }
    }

    return min;
}

int getMax(int *m, int n) {
    int max = m[0];

    for (int i = 0; i < n; i++) {
        if (max < m[i]) {
            max = m[i];
        }
    }

    return max;
}

void insertionSortRowsMatrixByRowCriteria(Matrix m, int (*criteria)(int *,
int)) {
    int *criteriaArray = (int *) malloc(sizeof(int) * m.nRows);
    for (int i = 0; i < m.nRows; i++) {
        criteriaArray[i] = criteria(m.values[i], m.nCols);
    }

    for (int i = 0; i < m.nRows; i++) {
        int minIndex = i;
        for (int j = i + 1; j < m.nRows; j++) {
            if (criteriaArray[j] < criteriaArray[minIndex]) {
                minIndex = j;
            }
        }

        if (minIndex != i) {
            swap(&criteriaArray[i], &criteriaArray[minIndex]);
            swapRows(m, i, minIndex);
        }
    }

    free(criteriaArray);
}
```

```

void selectionSortColsMatrixByColCriteria(Matrix m, int (*criteria)(int *,
int)) {
    int *criteriaArray = (int *) malloc(sizeof(int) * m.nCols);
    int *col = (int *) malloc(sizeof(int) * m.nCols);
    for (int j = 0; j < m.nCols; j++) {
        for (int i = 0; i < m.nRows; i++) {
            col[i] = m.values[i][j];
        }
        criteriaArray[j] = criteria(col, m.nCols);
    }

    for (int i = 0; i < m.nCols; i++) {
        int minIndex = i;
        for (int j = i + 1; j < m.nCols; j++) {
            if (criteriaArray[j] < criteriaArray[minIndex]) {
                minIndex = j;
            }

            if (minIndex != i) {
                swap(&criteriaArray[i], &criteriaArray[minIndex]);
                swapColumns(m, i, minIndex);
            }
        }

        free(col);
        free(criteriaArray);
    }
}

```

6. В библиотеке matrix реализуйте следующие функции-предикаты:

```
//6
bool isSquareMatrix(Matrix m) {
    return m.nRows == m.nCols ? true : false;
}

bool areTwoMatricesEqual(Matrix m1, Matrix m2) {
    if (m1.nRows != m2.nRows || m1.nCols != m2.nCols) {
        return false;
    }

    for (int i = 0; i < m1.nRows; i++) {
        if (memcmp(m1.values[i], m2.values[i], sizeof(int *) + 1) != 0) {
            return false;
        }
    }

    return true;
}

bool isEMatrix(Matrix m) {
    if (m.nRows != m.nCols) {
        return false;
    }

    int n = m.nRows;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if ((i == j) != m.values[i][j]) {
                return false;
            }
        }
    }

    return true;
}

bool isSymmetricMatrix(Matrix m) {
    if (m.nCols != m.nRows) {
        return false;
    }

    int n = m.nRows;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (m.values[i][j] != m.values[j][i]) {
                return false;
            }
        }
    }

    return true;
}
```

7. В библиотеке matrix реализуйте следующие функции преобразования матриц:

```
//7
void transposeSquareMatrix(Matrix m) {
    assert(isSquareMatrix(m));

    for (int i = 0; i < m.nRows; i++) {
        for (int j = i; j < m.nRows; j++) {
            int t = m.values[i][j];
            m.values[i][j] = m.values[j][i];
            m.values[j][i] = t;
        }
    }
}

void transposeMatrix(Matrix *m) {
    Matrix t = getMatrix(m->nCols, m->nRows);

    for (int i = 0; i < m->nRows; i++) {
        for (int j = 0; j < m->nCols; j++) {
            t.values[j][i] = m->values[i][j];
        }
    }

    freeMatrix(*m);
    *m = t;
}
```

8. В библиотеке matrix реализуйте функции для поиска минимального и максимального элемента матрицы

```
//8
Position getMinValuePos(Matrix m) {
    int iMin = 0;
    int jMin = 0;

    for (int i = 0; i < m.nRows; i++) {
        for (int j = 0; j < m.nCols; j++) {
            if (m.values[i][j] < m.values[iMin][jMin]) {
                iMin = i;
                jMin = j;
            }
        }
    }

    return (Position) {iMin, jMin};
}

Position getMaxValuePos(Matrix m) {
    int iMax = 0;
    int jMax = 0;

    for (int i = 0; i < m.nRows; i++) {
        for (int j = 0; j < m.nCols; j++) {
            if (m.values[i][j] > m.values[iMax][jMax]) {
                iMax = i;
                jMax = j;
            }
        }
    }

    return (Position) {iMax, jMax};
}
```


9. Дополните библиотеку функциями для тестирования:

```
//9
Matrix createMatrixFromArray(const int *a, int nRows, int nCols) {
    Matrix m = getMatrix(nRows, nCols);

    int k = 0;
    for (int i = 0; i < nRows; i++)
        for (int j = 0; j < nCols; j++)
            m.values[i][j] = a[k++];

    return m;
}

Matrix *createArrayOfMatrixFromArray(const int *values, size_t nMatrices,
size_t nRows, size_t nCols) {
    Matrix *ms = getArrayOfMatrices(nMatrices, nRows, nCols);
    int l = 0;
    for (size_t k = 0; k < nMatrices; k++)
        for (size_t i = 0; i < nRows; i++)
            for (size_t j = 0; j < nCols; j++)
                ms[k].values[i][j] = values[l++];

    return ms;
}
```

Тесты функций:

```
#include <stdio.h>
#include <assert.h>
#include "directoryOfMatrix/matrix.h"

void test_swapRowsMatrix() {
    Matrix m1 = createMatrixFromArray((int[]) {
        1,5,4,
        1,1,2,
        4,3,2
    },3, 3);

    swapRows(m1,0,1);
    outputMatrix(m1);

    Matrix m2 = createMatrixFromArray((int[]) {
        1,1,2,
        1,5,4,
        4,3,2
    },3, 3);

    outputMatrix(m2);
    assert(areTwoMatricesEqual(m1,m2));
}

void test_swapColumns() {
    Matrix m1 = createMatrixFromArray((int[]) {
        1,5,4,
        1,1,2,
        4,3,2,
        1,2,3
    },4, 3);

    swapColumns(m1,0,1);

    Matrix m2 = createMatrixFromArray((int[]) {
        5,1,4,
        1,1,2,
        3,4,2,
        2,1,3
    },4, 3);

    assert(areTwoMatricesEqual(m1,m2));
}

void test_sortRowsMatrixByRow() {
    Matrix m1 = createMatrixFromArray((int[]) {
        1,5,4,
        1,1,2,
        4,3,2,
        1,2,3
    },4, 3);

    insertionSortRowsMatrixByRowCriteria(m1, getSum);

    Matrix m2 = createMatrixFromArray((int[]) {
        1,1,2,
        1,2,3,
        4,3,2,
        1,5,4
    },4, 3);

    assert(areTwoMatricesEqual(m1,m2));
}
```

```

void test_sortColsMatrix() {
    Matrix m1 = createMatrixFromArray((int[]) {
        1,5,4,
        1,1,2,
        4,3,2,
        1,2,3
    },4, 3);

    selectionSortColsMatrixByColCriteria(m1,getSum);

    Matrix m2 = createMatrixFromArray((int[]) {
        1,4,5,
        1,2,1,
        4,2,3,
        1,3,2
    },4, 3);

    assert(areTwoMatricesEqual(m1,m2));
}

void test_squareMatrix() {
    Matrix m = createMatrixFromArray((int[]) {
        1,5,4,
        1,1,2,
        4,3,2
    },3, 3);

    assert(isSquareMatrix(m));
}

void test_twoMatricesEqual() {
    Matrix m1 = createMatrixFromArray((int[]) {
        1,5,4,
        1,1,2,
        4,3,2
    },3, 3);

    Matrix m2 = createMatrixFromArray((int[]) {
        1,5,4,
        1,1,2,
        4,3,2
    },3, 3);

    assert(areTwoMatricesEqual(m1,m2));
}

void test_isEMatrix() {
    Matrix m = createMatrixFromArray((int[]) {
        1,0,0,
        0,1,0,
        0,0,1
    },3, 3);

    assert(isEMatrix(m));
}

void test_isSymmetricMatrix() {
    Matrix m = createMatrixFromArray((int[]) {
        1,2,3,
        2,2,3,
        3,3,4
    },3, 3);

    assert(isSymmetricMatrix(m));
}

```

```

void test_transposeSquareMatrix() {
    Matrix m1 = createMatrixFromArray((int[]) {
        1,5,4,
        1,1,2,
        4,3,2
    },3, 3);

    transposeSquareMatrix(m1);

    Matrix m2 = createMatrixFromArray((int[]) {
        1,1,4,
        5,1,3,
        4,2,2
    },3, 3);

    assert(areTwoMatricesEqual(m1,m2));
}

void test_transposeMatrix() {
    Matrix m1 = createMatrixFromArray((int[]) {
        1,5,4,
        1,1,2,
        4,3,2,
        1,2,3
    },4, 3);

    transposeMatrix(&m1);

    Matrix m2 = createMatrixFromArray((int[]) {
        1,1,4,1,
        5,1,3,2,
        4,2,2,3
    },3, 4);

    assert(areTwoMatricesEqual(m1,m2));
}

```

Заголовочный файл:

```
#ifndef LABS_MATRIX_H
#define LABS_MATRIX_H
#include <stdbool.h>

//1
typedef struct matrix {
    int **values; // элементы матрицы
    int nRows; // количество рядов
    int nCols; // количество столбцов
} Matrix;

typedef struct position {
    int rowIndex;
    int colIndex;
} Position;

//2
Matrix getMatrix(int nRows, int nCols);
Matrix *getArrayOfMatrices(int numMatrix, int nRows, int nCols);
void freeMatrix(Matrix m);
void freeMemMatrices(Matrix *m, int numMatrix);

//3
void inputMatrix(Matrix m);
void outputMatrix(Matrix m);
void inputMatrices(Matrix *m, int n);
void outputMatrices(Matrix *m, int n);

//4
void swapRows(Matrix m, int i1, int i2);
void swapColumns(Matrix m, int j1, int j2);

//5
int getSum(int *a, int n);
int getMin(int *m, int n);
int getMax(int *m, int n);
void insertionSortRowsMatrixByRowCriteria(Matrix m, int (*criteria)(int*,
int));
void selectionSortColsMatrixByColCriteria(Matrix m, int (*criteria)(int *,
int));

//6
bool isSquareMatrix(Matrix m);
bool isEMatrix(Matrix m);
bool areTwoMatricesEqual(Matrix m1, Matrix m2);
bool isSymmetricMatrix(Matrix m);

//7
void transposeSquareMatrix(Matrix m);
void transposeMatrix(Matrix *m);

//8
Position getMinValuePos(Matrix m);
Position getMaxValuePos(Matrix m);

//9
Matrix createMatrixFromArray(const int *a, int nRows, int nCols);
#endif //LABS_MATRIX_H
```